

Cronómetro de Reacción - PIC16F887

Código comentado línea a línea + resumen de lógica

Resumen de lógica del sistema

Descripción general

El sistema mide el tiempo de reacción de un paciente usando un PIC16F887 a 4MHz. El operador elige una categoría con un potenciómetro, el circuito espera que el rango sea estable 2 segundos, avisa "LISTO" por UART, y el paciente debe presionar el botón RB0 lo más rápido posible. El tiempo se muestra en dos displays de 7 segmentos y se envía por puerto serie.

Categorías y límites de tiempo

Posición potenciómetro	LED	Categoría	Límite	ADC (ADRESH)
Mínimo (~0V)	Rojo (RA3)	Joven sano	250 ms	0 - 85
Medio (~2.5V)	Verde (RA4)	Adulto mayor	500 ms	86 - 170
Máximo (~5V)	Amarillo (RA5)	Paciente ACV	750 ms	171 - 255

Flujo completo de operación

ENCENDER circuito

|

v

LOOP: leer ADC continuamente

|

+-- Rango cambia -> reiniciar ESTABLE_COUNT y ESTABLE_SEC

|

+-- Rango estable -> cada 5ms (Timer0 ISR):

ESTABLE_COUNT++ hasta 200 (= 1 segundo)

Al llegar a 200 -> ESTABLE_SEC++, reiniciar ESTABLE_COUNT

Al llegar ESTABLE_SEC = 2 -> ARRANCAR_PRUEBA()

|

v

Envía "LISTO\r\n" por UART

Activa JUGANDO=1

|

v

Timer0 cada 10ms: incrementa NUM_UNI/NUM_DEC y CENTI_COUNT

RB1 parpadea cada 20ms (indicador visual de prueba activa)

|

+-----+-----+

|

Paciente presiona RB0
(antes del límite)

|

v

CONGELAR_TIEMPO()

JUGANDO=0

TERMINADO=1

RB1 fijo encendido

TX_NORMAL_POR_RANGO()

|

+-----+-----+

v

|

CENTI_COUNT == LIMITE
(tiempo agotado)

|

v

TX_EXCEDIDO_POR_RANGO()

JUGANDO=0, TERMINADO=1

RB1 fijo encendido

```

Display muestra tiempo congelado
UART muestra resultado + tiempo
    |
    v
Presionar RB0 nuevamente -> RESET_TOTAL()
Vuelve al estado inicial

```

Temporización (base de tiempo)

- **Oscilador:** 4 MHz interno -> ciclo de instrucción = 1 us
 - **Timer0:** Prescaler 1:64, precarga en 178 -> desborda cada ~**5 ms**
 - **Multiplexado de displays:** alterna decenas/unidades cada **5 ms** (imperceptible para el ojo)
 - **Parpadeo LED RB1:** cada 4 ticks x 5 ms = **20 ms**
 - **Unidad de tiempo display:** cada 2 ticks x 5 ms = **10 ms** (1 centésima de segundo)
 - **Estabilidad de rango:** 200 ticks x 5 ms = **1 segundo** x 2 = **2 segundos**
-

Estructura de memoria RAM

Banco compartido 0x70-0x7F (accesible desde cualquier banco)

Variable	Dirección	Función
W_TEMP	0x70	Guarda W al entrar a la ISR
STATUS_TEMP	0x71	Guarda STATUS al entrar a la ISR
PCLATH_TEMP	0x72	Guarda PCLATH al entrar a la ISR
DISP_DEC	0x73	Patrón 7-seg de las decenas
DISP_UNI	0x74	Patrón 7-seg de las unidades
NUM_DEC	0x75	Valor numérico de las decenas (0-9)
NUM_UNI	0x76	Valor numérico de las unidades (0-9)
MUX_FLAG	0x77	0=mostrar decenas, 1=mostrar unidades
JUGANDO	0x78	bit0=1 mientras el cronómetro corre
TERMINADO	0x79	bit0=1 cuando la prueba terminó
TICK_MUX	0x7A	Contador de ticks para parpadeo LED
TICK_CENTI	0x7B	Contador de ticks para la centésima
CENTI_COUNT	0x7C	Centésimas acumuladas (compara con LIMITE)
LIMITE	0x7D	Máximo de centésimas (25 / 50 / 75)
RANGO_PREV	0x7E	Último rango leído del ADC (1 / 2 / 3)
ESTABLE_COUNT	0x7F	Ticks de estabilidad dentro del segundo

Banco 0 GPR 0x20-0x22

Variable	Dirección	Función
TX_IDX	0x20	Índice de carácter para envío de cadenas UART
TX_DATA	0x21	Byte temporal para TX_BYTE
ESTABLE_SEC	0x22	Segundos completos de estabilidad (0-2)

Tablas de cadenas en memoria de programa

Las cadenas están en páginas altas de la memoria de programa para no solaparse con el código. Cada tabla usa ADDWF PCL,F para indexar directamente:

Tabla	Dirección ORG	Contenido
STR_JOVEN_NORMAL	0x200	Joven \ Prueba aprobada \
STR_JOVEN_EXCEDIDO	0x240	Joven \ Prueba desaprobada \
STR_ADULTO_NORMAL	0x280	Adulto mayor \ Prueba aprobada \
STR_ADULTO_EXCEDIDO	0x2C0	Adulto mayor \ Prueba desaprobada \
STR_ACV_NORMAL	0x300	Paciente ACV \ Prueba aprobada \
STR_ACV_EXCEDIDO	0x340	Paciente ACV \ Prueba desaprobada \

Ejemplo de salida UART al presionar el botón en el momento justo:

LISTO

Joven | Prueba aprobada | 18

(donde 18 = 18 centésimas x 10 ms = 180 ms de tiempo de reacción)

Conexiones de hardware

Pin PIC	Función
RA0	Digit-select display DECENAS
RA1	Digit-select display UNIDADES
RA2 (AN2)	Entrada analógica del potenciómetro
RA3	LED ROJO (Joven)
RA4	LED VERDE (Adulto mayor)
RA5	LED AMARILLO (Paciente ACV)
RB0	Botón de reacción (interrupción externa)
RB1	LED indicador (parpadea durante prueba)
RC6 (TX)	UART -> PC (9600,8N1)
RD0-RD6	Segmentos a-g del display

Código comentado línea a línea

```
; Cronómetro de Reacción - PIC16F887 - 4MHz interno

LIST      P=16F887      ; le dice al ensamblador que el micro es PIC16F887
INCLUDE   <P16F887.INC> ; incluye nombres de registros (PORTA, INTCON, etc.)

; bits de configuración grabados en el PIC:
; _INTRC_OSC_NOCLKOUT -> oscilador interno 4MHz, no saca reloj por pin
; _WDT_OFF            -> watchdog timer apagado (no resetea solo)
; _PWRTE_OFF          -> sin retardo al encender
; _MCLRE_OFF          -> pin MCLR deshabilitado (evita resets por ruido)
; _LVP_OFF            -> solo programación de alto voltaje
__CONFIG _CONFIG1, _INTRC_OSC_NOCLKOUT & _WDT_OFF & _PWRTE_OFF & _MCLRE_OFF & _LVP_OFF

; =====
; VARIABLES RAM - banco compartido 0x70-0x7F
; Accesibles desde CUALQUIER banco sin cambiar banco.
; Son exactamente 16 bytes (límite del bloque compartido).
; =====
CBLOCK    0x70
    W_TEMP      ; guarda W al entrar a la interrupción (ISR)
    STATUS_TEMP ; guarda STATUS al entrar a la ISR
    PCLATH_TEMP ; guarda PCLATH al entrar a la ISR
    DISP_DEC    ; patrón 7-seg del dígito DECENAS (sale a PORTD)
    DISP_UNI    ; patrón 7-seg del dígito UNIDADES (sale a PORTD)
    NUM_DEC     ; valor numérico de las decenas (0-9)
    NUM_UNI     ; valor numérico de las unidades (0-9)
    MUX_FLAG    ; flag de multiplexado: 0=decenas, 1=unidades
    JUGANDO     ; bit0=1 mientras el cronómetro está corriendo
    TERMINADO   ; bit0=1 cuando la prueba terminó (display congelado)
    TICK_MUX    ; contador de ticks para parpadeo del LED RB1 (0-3)
    TICK_CENTI  ; contador de ticks para formar 10ms (0-1)
    CENTI_COUNT ; centésimas de segundo acumuladas
    LIMITE      ; límite máximo según categoría (25/50/75)
    RANGO_PREV  ; último rango leído del ADC (1=rojo,2=verde,3=amarillo)
    ESTABLE_COUNT ; ticks de 5ms dentro del segundo actual (0-199)
ENDC

; Banco 0 GPR 0x20-0x22 - solo accesibles desde banco 0
CBLOCK    0x20
    TX_IDX      ; índice del carácter actual al enviar cadena UART
    TX_DATA     ; byte temporal para TX_BYTE
    ESTABLE_SEC ; segundos completos de estabilidad contados (0-2)
ENDC

; =====
; VECTORES DE INTERRUPTIÓN
; =====
ORG       0x0000      ; al encender, el PIC salta a esta dirección
GOTO      INICIO      ; va al programa principal

ORG       0x0004      ; cualquier interrupción salta aquí
GOTO      ISR          ; va a la rutina de servicio de interrupción

; =====
; TABLA 7 SEGMENTOS - convierte dígito (0-9) a patrón
```

```

; Cátodo común: bit=1 enciende el segmento
; =====
TABLA:
    ANDLW    0x0F      ; limita W a rango 0-15 (máscara)
    ADDWF    PCL, F    ; suma W al PC -> salta a la RETLW del dígito
    RETLW    b'00111111' ; 0 -> segmentos a,b,c,d,e,f
    RETLW    b'00000110' ; 1 -> b,c
    RETLW    b'01011011' ; 2 -> a,b,d,e,g
    RETLW    b'01001111' ; 3 -> a,b,c,d,g
    RETLW    b'01100110' ; 4 -> b,c,f,g
    RETLW    b'01101101' ; 5 -> a,c,d,f,g
    RETLW    b'01111101' ; 6 -> a,c,d,e,f,g
    RETLW    b'00000111' ; 7 -> a,b,c
    RETLW    b'01111111' ; 8 -> todos los segmentos
    RETLW    b'01101111' ; 9 -> a,b,c,d,f,g

; =====
; INICIO - configuración de hardware (se ejecuta solo una vez)
; =====
INICIO:
    ; --- Oscilador interno 4MHz ---
    BANKSEL OSCCON
    MOVLW    b'01100100' ; IRCF=110 -> 4MHz, SCS=0 -> usa config bits
    MOVWF    OSCCON

    ; --- Solo AN2 (RA2) es analógico, el resto digital ---
    BANKSEL ANSEL
    MOVLW    b'00000100' ; bit2=1 -> AN2 analógico
    MOVWF    ANSEL
    CLRF     ANSELH      ; ANSELH=0 -> PORTB/PORTC son digitales

    ; --- Dirección de pines ---
    BANKSEL TRISA
    MOVLW    b'00000100' ; RA2=entrada(pot), resto=salidas(displays/LEDs)
    MOVWF    TRISA
    BANKSEL TRISB
    MOVLW    b'00000001' ; RB0=entrada(botón), RB1=salida(LED)
    MOVWF    TRISB
    BANKSEL TRISD
    CLRF     TRISD      ; PORTD todo salidas (segmentos del display)

    BANKSEL TRISC
    BCF      TRISC, 6    ; RC6=salida (TX UART)
    BSF      TRISC, 7    ; RC7=entrada (RX UART)

    ; --- Limpiar puertos de salida ---
    BANKSEL PORTA
    CLRF     PORTA      ; apaga LEDs y digit-select
    BANKSEL PORTB
    CLRF     PORTB      ; apaga LED RB1
    BANKSEL PORTD
    CLRF     PORTD      ; apaga segmentos

    ; --- ADC: canal AN2, Fosc/8, resultado en ADRESH ---
    BANKSEL ADCON1
    CLRF     ADCON1     ; Vref=VDD/VSS, resultado justificado izquierda

```

```

BANKSEL ADCON0
MOVLW    b'01001001' ; ADCS=01(Fosc/8), CHS=010(AN2), ADON=1
MOVWF    ADCON0

; --- Timer0 + configuración de RBO ---
BANKSEL OPTION_REG
MOVLW    b'00000101' ; INTEDG=0->flanco bajada RBO, RBPU=0->pull-ups ON,
                    ; TOCS=0->Fosc/4, PS=101->prescaler 1:64
MOVWF    OPTION_REG

BANKSEL TMRO
MOVLW    d'178'      ; precarga: 256-178=78 ticks x 64us ~ 5ms
MOVWF    TMRO

; --- UART 9600 baudios, 8 bits, sin paridad ---
BANKSEL SPBRG
MOVLW    d'25'       ; baud=4MHz/(16x26)=9615~9600 con BRGH=1
MOVWF    SPBRG
BANKSEL RCSTA
MOVLW    b'10000000' ; SPEN=1 -> habilita módulo USART
MOVWF    RCSTA
BANKSEL TXSTA
MOVLW    b'00100100' ; TXEN=1->habilita TX, BRGH=1->alta velocidad
MOVWF    TXSTA

; --- Inicializar todas las variables en 0 ---
BANKSEL PORTA ; vuelve a banco 0 (TX_IDX, TX_DATA, ESTABLE_SEC aquí)
CLRF      TX_IDX
CLRF      TX_DATA
CLRF      NUM_DEC
CLRF      NUM_UNI
CLRF      MUX_FLAG
CLRF      JUGANDO
CLRF      TERMINADO
CLRF      TICK_MUX
CLRF      TICK_CENTI
CLRF      CENTI_COUNT
CLRF      LIMITE
CLRF      RANGO_PREV
CLRF      ESTABLE_COUNT
CLRF      PCLATH_TEMP
CLRF      PCLATH

CALL      CONV_DISPLAYS ; convierte 00 a patrones 7-seg -> muestra "00"

; --- Habilitar interrupciones ---
BANKSEL INTCON
MOVLW    b'10110000' ; GIE=1->global ON, TOIE=1->Timer0 ON, INTE=1->RBO ON
MOVWF    INTCON

; =====
; LOOP PRINCIPAL - se ejecuta indefinidamente
; =====
LOOP:
    BTFSC    TERMINADO, 0 ; si TERMINADO.bit0=1 -> prueba congelada
    GOTO     LOOP        ; quedarse esperando IRQ del botón RBO

```

```

BTFSC   JUGANDO, 0    ; si JUGANDO.bit0=1 -> cronómetro corriendo
GOTO    LOOP          ; quedarse esperando IRQ RBO o Timer0

```

```

; ESPERANDO: leer ADC continuamente, detectar cambio de rango
CALL    LEER_ADC      ; lee pot, enciende LED, retorna rango(1/2/3) en W
SUBWF   RANGO_PREV, W ; W = RANGO_PREV - W(rango nuevo)
BTFSS   STATUS, Z     ; si Z=0 -> rango diferente -> cambio detectado
GOTO    RANGO_CAMBIO
GOTO    LOOP          ; mismo rango -> seguir monitoreando

```

RANGO_CAMBIO:

```

CALL    LEER_ADC      ; lee el ADC de nuevo para obtener nuevo rango en W
MOVWF   RANGO_PREV    ; guarda el nuevo rango
CLRF    ESTABLE_COUNT ; reinicia contador -> hay que esperar estabilidad de nuevo
GOTO    LOOP

```

```

; =====
; LEER_ADC - dispara conversión y retorna rango en W (1/2/3)
;           también enciende el LED correspondiente
; =====

```

LEER_ADC:

```

BANKSEL ADCON0
BSF     ADCON0, GO    ; inicia conversión ADC (bit GO=1)

```

WAIT_ADC:

```

BTFSC   ADCON0, GO    ; espera a que GO se limpie (conversión terminada)
GOTO    WAIT_ADC

```

```

MOVF    ADRESH, W     ; W = byte alto del resultado ADC (0-255)
SUBLW   d'85'         ; W = 85 - W
BTFSS   STATUS, C     ; si Carry=0 -> ADC era >85 -> ir a verde
GOTO    ADC_VERDE

```

```

; ADC 0-85 -> Rango 1 (JOVEN) -> LED ROJO en RA3

```

```

BANKSEL PORTA
BSF     PORTA, 3      ; enciende LED rojo
BCF     PORTA, 4      ; apaga LED verde
BCF     PORTA, 5      ; apaga LED amarillo
MOVLW   d'1'         ; retorna rango 1
RETURN

```

ADC_VERDE:

```

MOVF    ADRESH, W     ; W = 170 - W
SUBLW   d'170'        ; W = 170 - W
BTFSS   STATUS, C     ; si Carry=0 -> ADC era >170 -> ir a amarillo
GOTO    ADC_AMARILLO

```

```

; ADC 86-170 -> Rango 2 (ADULTO MAYOR) -> LED VERDE en RA4

```

```

BANKSEL PORTA
BCF     PORTA, 3
BSF     PORTA, 4      ; enciende LED verde
BCF     PORTA, 5
MOVLW   d'2'
RETURN

```

ADC_AMARILLO:


```

; ADC 171-255 -> Rango 3 (PACIENTE ACV) -> LED AMARILLO en RA5
BANKSEL PORTA
BCF     PORTA, 3
BCF     PORTA, 4
BSF     PORTA, 5      ; enciende LED amarillo
MOVLW   d'3'
RETURN

; =====
; CARGAR_LIMITE - carga en LIMITE el máx. de centésimas
; según RANGO_PREV (1->25, 2->50, 3->75)
; =====
CARGAR_LIMITE:
    MOVF   RANGO_PREV, W
    SUBLW   d'1'
    BTFSS   STATUS, Z      ; si era rango 1 (Joven)
    GOTO    CL_VERDE
    MOVLW   d'25'          ; 25 x 10ms = 250ms
    MOVWF   LIMITE
    RETURN

CL_VERDE:
    MOVF   RANGO_PREV, W
    SUBLW   d'2'
    BTFSS   STATUS, Z      ; si era rango 2 (Adulto mayor)
    GOTO    CL_AMARILLO
    MOVLW   d'50'          ; 50 x 10ms = 500ms
    MOVWF   LIMITE
    RETURN

CL_AMARILLO:
    MOVLW   d'75'          ; 75 x 10ms = 750ms
    MOVWF   LIMITE
    RETURN

; =====
; CONV_DISPLAYS - convierte NUM_DEC/NUM_UNI a patrones 7-seg
; =====
CONV_DISPLAYS:
    CLRF    PCLATH          ; TABLA está en página 0 (dir < 0x100)
    MOVF    NUM_DEC, W
    CALL    TABLA           ; convierte decena a patrón 7-seg
    MOVWF   DISP_DEC
    MOVF    NUM_UNI, W
    CALL    TABLA           ; convierte unidad a patrón 7-seg
    MOVWF   DISP_UNI
    RETURN

; =====
; RESET_TOTAL - reinicia todo al estado inicial
; =====
RESET_TOTAL:
    CLRF    JUGANDO
    CLRF    TERMINADO
    CLRF    NUM_DEC
    CLRF    NUM_UNI
    CLRF    CENTI_COUNT
    CLRF    TICK_CENTI

```

```

CLRFB    TICK_MUX
CLRFB    ESTABLE_COUNT
CLRFB    LIMITE
CLRFB    RANGO_PREV    ; fuerza releer el pot en el próximo LOOP
BANKSEL PORTB
BCFB    PORTB, 1    ; apaga LED indicador RB1
BANKSEL PORTD
CLRFB    PORTD    ; apaga segmentos
BANKSEL PORTA
BCFB    PORTA, 0    ; apaga digit-select decenas
BCFB    PORTA, 1    ; apaga digit-select unidades
CALL    CONV_DISPLAYS
RETURN

```

```

; =====
; ARRANCAR_PRUEBA - inicia cronómetro, envía "LISTO" por UART
; Se llama desde ISR cuando el rango estuvo estable 2 segundos
; =====

```

ARRANCAR_PRUEBA:

```

CALL    CARGAR_LIMITE ; carga el límite según categoría
CLRFB    NUM_DEC    ; tiempo en 00
CLRFB    NUM_UNI
CLRFB    CENTI_COUNT
CLRFB    TICK_CENTI
CLRFB    TICK_MUX
CALL    CONV_DISPLAYS ; actualiza displays con 00
; Envía "LISTO\r\n" por UART
MOVLW    'L'
CALL    TX_BYTE
MOVLW    'I'
CALL    TX_BYTE
MOVLW    'S'
CALL    TX_BYTE
MOVLW    'T'
CALL    TX_BYTE
MOVLW    'O'
CALL    TX_BYTE
MOVLW    0x0D    ; \r (carriage return)
CALL    TX_BYTE
MOVLW    0x0A    ; \n (line feed)
CALL    TX_BYTE
BSFB    JUGANDO, 0    ; activa flag -> Timer0 empieza a contar tiempo
RETURN

```

```

; =====
; TX_DIGITOS_CRLF - envía NUM_DEC y NUM_UNI como ASCII + \r\n
; Ejemplo: NUM_DEC=1, NUM_UNI=8 -> envía "18\r\n"
; =====

```

TX_DIGITOS_CRLF:

```

CLRFB    PCLATH
MOVF    NUM_DEC, W
ADDLW    '0'    ; convierte número a ASCII (0->'0', 1->'1', etc.)
CALL    TX_BYTE
MOVF    NUM_UNI, W
ADDLW    '0'
CALL    TX_BYTE

```

```

    MOVLW    0x0D        ; \r
    CALL     TX_BYTE
    MOVLW    0x0A        ; \n
    CALL     TX_BYTE
    RETURN

; =====
; TX_BYTE - envía un byte por UART (byte a enviar en W)
; =====
TX_BYTE:
    MOVWF    TX_DATA      ; guarda W en variable temporal
TX_BYTE_WAIT:
    BANKSEL  TXSTA
    BTFSS    TXSTA, TRMT  ; espera a que el transmisor esté libre (TRMT=1)
    GOTO     TX_BYTE_WAIT
    BANKSEL  TXREG
    MOVF     TX_DATA, W    ; recupera el byte
    MOVWF    TXREG        ; carga en registro TX -> sale por RC6
    RETURN

; =====
; TX_NORMAL_POR_RANGO - envía "prueba aprobada" + tiempo
; =====
TX_NORMAL_POR_RANGO:
    MOVF     RANGO_PREV, W
    SUBLW    d'1'
    BTFSS    STATUS, Z
    GOTO     TNR_R2
    CALL     TX_JOVEN_NORMAL      ; "Joven / Prueba aprobada / "
    CALL     TX_DIGITOS_CRLF     ; "18\r\n" (ejemplo)
    RETURN
TNR_R2:
    MOVF     RANGO_PREV, W
    SUBLW    d'2'
    BTFSS    STATUS, Z
    GOTO     TNR_R3
    CALL     TX_ADULTO_NORMAL    ; "Adulto mayor / Prueba aprobada / "
    CALL     TX_DIGITOS_CRLF
    RETURN
TNR_R3:
    CALL     TX_ACV_NORMAL      ; "Paciente ACV / Prueba aprobada / "
    CALL     TX_DIGITOS_CRLF
    RETURN

; =====
; TX_EXCEDIDO_POR_RANGO - envía "prueba desaprobada" + tiempo
; =====
TX_EXCEDIDO_POR_RANGO:
    MOVF     RANGO_PREV, W
    SUBLW    d'1'
    BTFSS    STATUS, Z
    GOTO     TER_R2
    CALL     TX_JOVEN_EXCEDIDO
    CALL     TX_DIGITOS_CRLF
    RETURN
TER_R2:

```

```

    MOVF    RANGO_PREV, W
    SUBLW   d'2'
    BTFSS   STATUS, Z
    GOTO    TER_R3
    CALL    TX_ADULTO_EXCEDIDO
    CALL    TX_DIGITOS_CRLF
    RETURN

TER_R3:
    CALL    TX_ACV_EXCEDIDO
    CALL    TX_DIGITOS_CRLF
    RETURN

; =====
; Rutinas TX de cadena - recorren la tabla hasta encontrar 0x00
; PCLATH se configura antes de cada CALL según la página de la tabla
; =====

TX_JOVEN_NORMAL:
    CLRF    TX_IDX
    MOVLW   0x02          ; tabla en 0x200 -> página 2
    MOVWF   PCLATH
TXJ_NRM_LP:
    MOVF    TX_IDX, W
    CALL    STR_JOVEN_NORMAL
    ANDLW   0xFF          ; actualiza flag Z
    BTFSC   STATUS, Z      ; si W=0 -> fin de cadena
    RETURN
    CALL    TX_BYTE
    INCF    TX_IDX, F
    GOTO    TXJ_NRM_LP

TX_JOVEN_EXCEDIDO:
    CLRF    TX_IDX
    MOVLW   0x02          ; tabla en 0x240 -> página 2
    MOVWF   PCLATH
TXJ_EXC_LP:
    MOVF    TX_IDX, W
    CALL    STR_JOVEN_EXCEDIDO
    ANDLW   0xFF
    BTFSC   STATUS, Z
    RETURN
    CALL    TX_BYTE
    INCF    TX_IDX, F
    GOTO    TXJ_EXC_LP

TX_ADULTO_NORMAL:
    CLRF    TX_IDX
    MOVLW   0x02          ; tabla en 0x280 -> página 2
    MOVWF   PCLATH
TXA_NRM_LP:
    MOVF    TX_IDX, W
    CALL    STR_ADULTO_NORMAL
    ANDLW   0xFF
    BTFSC   STATUS, Z
    RETURN
    CALL    TX_BYTE

```

```

    INCF    TX_IDX, F
    GOTO    TXA_NRM_LP

TX_ADULTO_EXCEDIDO:
    CLRF    TX_IDX
    MOVLW   0x02          ; tabla en 0x2C0 -> página 2
    MOVWF   PCLATH
TXA_EXC_LP:
    MOVF    TX_IDX, W
    CALL    STR_ADULTO_EXCEDIDO
    ANDLW   0xFF
    BTFSC   STATUS, Z
    RETURN
    CALL    TX_BYTE
    INCF    TX_IDX, F
    GOTO    TXA_EXC_LP

TX_ACV_NORMAL:
    CLRF    TX_IDX
    MOVLW   0x03          ; tabla en 0x300 -> página 3
    MOVWF   PCLATH
TXACV_NRM_LP:
    MOVF    TX_IDX, W
    CALL    STR_ACV_NORMAL
    ANDLW   0xFF
    BTFSC   STATUS, Z
    RETURN
    CALL    TX_BYTE
    INCF    TX_IDX, F
    GOTO    TXACV_NRM_LP

TX_ACV_EXCEDIDO:
    CLRF    TX_IDX
    MOVLW   0x03          ; tabla en 0x340 -> página 3
    MOVWF   PCLATH
TXACV_EXC_LP:
    MOVF    TX_IDX, W
    CALL    STR_ACV_EXCEDIDO
    ANDLW   0xFF
    BTFSC   STATUS, Z
    RETURN
    CALL    TX_BYTE
    INCF    TX_IDX, F
    GOTO    TXACV_EXC_LP

; =====
; ISR - Rutina de Servicio de Interrupción
; Se ejecuta al dispararse IRQ de RBO o Timer0
; =====
ISR:
    ; Guardar contexto (W, STATUS, PCLATH) para no corromper el programa
    MOVWF   W_TEMP          ; guarda W
    SWAPF   STATUS, W        ; swap nibbles de STATUS (no afecta flags)
    MOVWF   STATUS_TEMP     ; guarda STATUS
    MOVF    PCLATH, W        ; guarda PCLATH (puede estar en página 2/3)
    MOVWF   PCLATH_TEMP

```

```

CLRf    PCLATH          ; vuelve a página 0 para ejecutar ISR

; --- 1. ¿Fue interrupción de RBO (botón)? ---
BANKSEL INTCON
BTFSS   INTCON, INTF     ; si INTF=0 -> no fue RBO
GOTO    CHECK_TMRO

BTFSC   TERMINADO, 0     ; si TERMINADO=1 -> botón = RESET
GOTO    HACER_RESET

BTFSC   JUGANDO, 0       ; si JUGANDO=1 -> botón = CONGELAR tiempo
GOTO    CONGELAR_TIEMPO

GOTO    CLEAR_INTF       ; si esperando -> ignorar botón

HACER_RESET:
CALL    RESET_TOTAL     ; reinicia todas las variables
WAIT_REL_RST:
BANKSEL PORTB
BTFSS   PORTB, 0        ; espera que el botón se suelte (RBO=1)
GOTO    WAIT_REL_RST     ; si sigue presionado, esperar (evita rebote)
GOTO    CLEAR_INTF

CONGELAR_TIEMPO:
CLRf    JUGANDO         ; detiene el cronómetro
BSF     TERMINADO, 0     ; marca prueba terminada
BANKSEL PORTB
BSF     PORTB, 1        ; enciende LED RB1 fijo
CALL    TX_NORMAL_POR_RANGO ; envía resultado + tiempo por UART
WAIT_REL_JUG:
BANKSEL PORTB
BTFSS   PORTB, 0        ; espera que el botón se suelte
GOTO    WAIT_REL_JUG

CLEAR_INTF:
BANKSEL INTCON
BCF     INTCON, INTF     ; limpia flag de IRQ RBO (obligatorio)
GOTO    FIN_ISR

; --- 2. ¿Fue interrupción del Timer0 (~5ms)? ---
CHECK_TMRO:
BTFSS   INTCON, TOIF     ; si TOIF=0 -> no fue Timer0
GOTO    FIN_ISR

BANKSEL TMRO
MOVLW   d'178           ; recarga Timer0 para próximo intervalo de ~5ms
MOVWF   TMRO

; A) MULTIPLEXADO DE DISPLAYS (alterna decenas/unidades cada 5ms)
BANKSEL PORTA
BCF     PORTA, 0        ; apaga digit-select decenas antes de cambiar
BCF     PORTA, 1        ; apaga digit-select unidades

MOVF    JUGANDO, F       ; ¿está jugando?
BTFSS   STATUS, Z
GOTO    HACER_MUX

```

```

MOVF    TERMINADO, F      ; ¿está terminado?
BTFSS   STATUS, Z
GOTO    HACER_MUX
GOTO    LOGICA_ESTABILIDAD ; ni jugando ni terminado -> no mostrar display

```

HACER_MUX:

```

BTFSC   MUX_FLAG, 0      ; si MUX_FLAG=1 -> turno de unidades
GOTO    MUX_UNI

```

MUX_DEC:

```

BANKSEL PORTD
MOVF    DISP_DEC, W      ; patrón de segmentos de las decenas
MOVWF   PORTD           ; envía a PORTD
BANKSEL PORTA
BSF     PORTA, 0         ; enciende digit-select decenas (RA0)
BSF     MUX_FLAG, 0      ; próxima vez: unidades
GOTO    LOGICA_ESTABILIDAD

```

MUX_UNI:

```

BANKSEL PORTD
MOVF    DISP_UNI, W      ; patrón de segmentos de las unidades
MOVWF   PORTD
BANKSEL PORTA
BSF     PORTA, 1         ; enciende digit-select unidades (RA1)
BCF     MUX_FLAG, 0      ; próxima vez: decenas

```

; B) LÓGICA DE ESTABILIDAD DEL RANGO (cuenta tiempo quieto)

LOGICA_ESTABILIDAD:

```

MOVF    JUGANDO, F
BTFSS   STATUS, Z
GOTO    LOGICA_JUEGO     ; si jugando -> saltar
MOVF    TERMINADO, F
BTFSS   STATUS, Z
GOTO    LOGICA_JUEGO     ; si terminado -> saltar

MOVF    RANGO_PREV, F
BTFSC   STATUS, Z        ; si RANGO_PREV=0 -> aún no se eligió categoría
GOTO    LOGICA_JUEGO

INCF    ESTABLE_COUNT, F ; suma 1 tick (5ms)
MOVLW   d'200'          ; 200 x 5ms = 1 segundo
SUBWF   ESTABLE_COUNT, W
BTFSS   STATUS, Z        ; ¿completó 1 segundo?
GOTO    LOGICA_JUEGO     ; no -> seguir esperando

CLRF    ESTABLE_COUNT    ; reinicia ticks para el próximo segundo
INCF    ESTABLE_SEC, F   ; suma 1 segundo estable
MOVLW   d'2'            ; queremos 2 segundos de estabilidad
SUBWF   ESTABLE_SEC, W
BTFSS   STATUS, Z        ; ¿completó 2 segundos?
GOTO    LOGICA_JUEGO     ; no -> seguir esperando

CALL    ARRANCAR_PRUEBA ; ¡2 segundos estables! -> arrancar prueba
GOTO    CLEAR_TOIF

```

; C) LÓGICA DE JUEGO (cronómetro corriendo)

LOGICA_JUEGO:

```

BTFSS  JUGANDO, 0      ; si no está jugando -> nada que hacer
GOTO   CLEAR_TOIF

```

```

; Parpadeo LED RB1 cada 4 ticks x 5ms = 20ms

```

```

INCF   TICK_MUX, F
MOVLW  d'4'
SUBWF  TICK_MUX, W
BTFSS  STATUS, Z      ; ¿llegó a 4?
GOTO   LOGICA_CENTI
CLRF   TICK_MUX
BANKSEL PORTB
MOVLW  b'00000010'    ; máscara bit RB1
XORWF  PORTB, F       ; toggle RB1 -> parpadeo

```

```

; Incrementar tiempo cada 2 ticks x 5ms = 10ms

```

LOGICA_CENTI:

```

INCF   TICK_CENTI, F
MOVLW  d'2'
SUBWF  TICK_CENTI, W
BTFSS  STATUS, Z      ; ¿pasaron 10ms?
GOTO   CLEAR_TOIF

```

```

CLRF   TICK_CENTI
INCF   CENTI_COUNT, F ; suma 1 centésima al tiempo transcurrido

```

```

; Incrementar dígito display

```

```

INCF   NUM_UNI, F
MOVLW  d'10'
SUBWF  NUM_UNI, W
BTFSS  STATUS, Z      ; ¿unidades llegaron a 10?
GOTO   CHK_LIMITE
CLRF   NUM_UNI        ; unidades -> 0
INCF   NUM_DEC, F     ; suma 1 a decenas
MOVLW  d'10'
SUBWF  NUM_DEC, W
BTFSS  STATUS, Z
GOTO   CHK_LIMITE
CLRF   NUM_DEC        ; decenas -> 0

```

CHK_LIMITE:

```

; ¿Se agotó el tiempo?

```

```

MOVF   LIMITE, W
SUBWF  CENTI_COUNT, W ; W = CENTI_COUNT - LIMITE
BTFSS  STATUS, Z      ; si no son iguales -> tiempo no agotado
GOTO   ACT_DISPLAYS

```

```

; Tiempo agotado -> congelar y enviar desaprobada

```

```

CLRF   JUGANDO
BSF    TERMINADO, 0
BANKSEL PORTB
BSF    PORTB, 1      ; LED fijo
CALL   TX_EXCEDIDO_POR_RANGO

```

ACT_DISPLAYS:

```

CALL   CONV_DISPLAYS ; actualiza patrones con tiempo actual

```



```

CLEAR_TOIF:
    BANKSEL INTCON
    BCF      INTCON, TOIF      ; limpia flag Timer0 (obligatorio)

FIN_ISR:
    ; Restaurar contexto (orden inverso al guardado)
    MOVF     PCLATH_TEMP, W
    MOVWF    PCLATH           ; restaura PCLATH
    SWAPF    STATUS_TEMP, W
    MOVWF    STATUS           ; restaura STATUS
    SWAPF    W_TEMP, F
    SWAPF    W_TEMP, W        ; restaura W sin afectar STATUS
    RETFIE      ; retorna y reactiva GIE automáticamente

; =====
; TABLAS DE CADENAS
; Cada tabla está fijada en una dirección con ORG.
; El primer RETLW (ANDLW+ADDWF) indexa la tabla.
; Terminan con RETLW 0x00 (fin de cadena).
; =====

    ORG      0x200      ; "Joven / Prueba aprobada / "
STR_JOVEN_NORMAL:
    ANDLW    0x1F
    ADDWF    PCL, F
    RETLW    'J'
    RETLW    'o'
    RETLW    'v'
    RETLW    'e'
    RETLW    'n'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    'P'
    RETLW    'r'
    RETLW    'u'
    RETLW    'e'
    RETLW    'b'
    RETLW    'a'
    RETLW    ' '
    RETLW    'a'
    RETLW    'p'
    RETLW    'r'
    RETLW    'o'
    RETLW    'b'
    RETLW    'a'
    RETLW    'd'
    RETLW    'a'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    0x00      ; fin de cadena

    ORG      0x240      ; "Joven / Prueba desaprobada / "
STR_JOVEN_EXCEDIDO:

```

```

ANDLW    0x1F
ADDWF    PCL, F
RETLW    'J'
RETLW    'o'
RETLW    'v'
RETLW    'e'
RETLW    'n'
RETLW    ' '
RETLW    '|'
RETLW    ' '
RETLW    'p'
RETLW    'r'
RETLW    'u'
RETLW    'e'
RETLW    'b'
RETLW    'a'
RETLW    ' '
RETLW    'd'
RETLW    'e'
RETLW    's'
RETLW    'a'
RETLW    'p'
RETLW    'r'
RETLW    'o'
RETLW    'b'
RETLW    'a'
RETLW    'd'
RETLW    'a'
RETLW    ' '
RETLW    '|'
RETLW    ' '
RETLW    0x00

```

```

ORG      0x280 ; "Adulto mayor / Prueba aprobada / "

```

```

STR_ADULTO_NORMAL:

```

```

ANDLW    0x3F
ADDWF    PCL, F
RETLW    'A'
RETLW    'd'
RETLW    'u'
RETLW    'l'
RETLW    't'
RETLW    'o'
RETLW    ' '
RETLW    'm'
RETLW    'a'
RETLW    'y'
RETLW    'o'
RETLW    'r'
RETLW    ' '
RETLW    '|'
RETLW    ' '
RETLW    'P'
RETLW    'r'
RETLW    'u'
RETLW    'e'

```

```

    RETLW    'b'
    RETLW    'a'
    RETLW    ' '
    RETLW    'a'
    RETLW    'p'
    RETLW    'r'
    RETLW    'o'
    RETLW    'b'
    RETLW    'a'
    RETLW    'd'
    RETLW    'a'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    0x00

    ORG      0x2C0    ; "Adulto mayor / Prueba desaprobada / "
STR_ADULTO_EXCEDIDO:
    ANDLW    0x3F
    ADDWF    PCL, F
    RETLW    'A'
    RETLW    'd'
    RETLW    'u'
    RETLW    'l'
    RETLW    't'
    RETLW    'o'
    RETLW    ' '
    RETLW    'm'
    RETLW    'a'
    RETLW    'y'
    RETLW    'o'
    RETLW    'r'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    'P'
    RETLW    'r'
    RETLW    'u'
    RETLW    'e'
    RETLW    'b'
    RETLW    'a'
    RETLW    ' '
    RETLW    'd'
    RETLW    'e'
    RETLW    's'
    RETLW    'a'
    RETLW    'p'
    RETLW    'r'
    RETLW    'o'
    RETLW    'b'
    RETLW    'a'
    RETLW    'd'
    RETLW    'a'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '

```

```
    RETLW    0x00

    ORG      0x300    ; "Paciente ACV / Prueba aprobada / "
STR_ACV_NORMAL:
    ANDLW    0x3F
    ADDWF    PCL, F
    RETLW    'P'
    RETLW    'a'
    RETLW    'c'
    RETLW    'i'
    RETLW    'e'
    RETLW    'n'
    RETLW    't'
    RETLW    'e'
    RETLW    ' '
    RETLW    'A'
    RETLW    'C'
    RETLW    'V'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    'P'
    RETLW    'r'
    RETLW    'u'
    RETLW    'e'
    RETLW    'b'
    RETLW    'a'
    RETLW    ' '
    RETLW    'a'
    RETLW    'p'
    RETLW    'r'
    RETLW    'o'
    RETLW    'b'
    RETLW    'a'
    RETLW    'd'
    RETLW    'a'
    RETLW    ' '
    RETLW    '|'
    RETLW    ' '
    RETLW    0x00

    ORG      0x340    ; "Paciente ACV / Prueba desaprobada / "
STR_ACV_EXCEDIDO:
    ANDLW    0x3F
    ADDWF    PCL, F
    RETLW    'P'
    RETLW    'a'
    RETLW    'c'
    RETLW    'i'
    RETLW    'e'
    RETLW    'n'
    RETLW    't'
    RETLW    'e'
    RETLW    ' '
    RETLW    'A'
    RETLW    'C'
```

```
RETLW  'V'
RETLW  ' '
RETLW  '|'
RETLW  ' '
RETLW  'P'
RETLW  'r'
RETLW  'u'
RETLW  'e'
RETLW  'b'
RETLW  'a'
RETLW  ' '
RETLW  'd'
RETLW  'e'
RETLW  's'
RETLW  'a'
RETLW  'p'
RETLW  'r'
RETLW  'o'
RETLW  'b'
RETLW  'a'
RETLW  'd'
RETLW  'a'
RETLW  ' '
RETLW  '|'
RETLW  ' '
RETLW  0x00      ; fin de cadena

END              ; fin del archivo fuente
```